

JOBSHEET 9

OVERLOADING AND OVERRIDING

1. Competence

After taking this subject, students are able to:

- a. Understand the concept of overloading and overriding,
- b. Understand the difference between overloading and overriding,
- c. Accuracy in identifying overriding and overloading methods
- d. Accuracy in practicing the instructions on the jobsheet
- e. Implement overloading and overriding methods.

2. Introduction

2.1 Overloading

is to rewrite a method with the same name in a class. The goal is to facilitate the use/summoning of methods with similar functionality. For the Overloading method declaration rule as follows:

- The name of the method must be the same.
- The list of parameters should be different.
- The return type can be the same, it can also be different.

There is a list of parameters on overloading can be seen as follows:

- The difference in the list of parameters does not only occur in the difference in the number of parameters, but also in the order of those parameters.
- For example, the following two parameters:
 - `Function_member (int x, string n)`
 - `Function_member (String n, int x)`
- The two parameters are also considered different from the list of parameters.
- The parameter list is not related to the naming of variables that exist in the parameters.
- For example, the following 2 parameter lists:
 - `function_member(int x)`
 - `function_member(int y)`

- The two parameter lists above are considered the same because the difference is only the naming of the variable parameters.

Overloading can also occur between the parent class and its subclasses if it meets all three overload requirements. There are several overloading rules, namely:

- Primitive widening conversion takes precedence over overloading over boxing and var args.
- We can't do the widening process from one wrapper type to another wrapper type (change the Integer to Long).
- We can't do the widening process and continue boxing (from int to long)
- We can do boxing followed by widening (int can be an Object via Integer)
- We can combine var args with either widening or boxing

2.2 Overriding

is a subclass that seeks to modify the behavior inherited from the superclass. The goal is that the subclass can have more specific behaviors so that it can be done by redeclaring the parent class's method in the subclass.

The declaration method in the subclass must be the same as the one in the super class.

Similarities to:

- Name
- Return type (for return type: class A or is a subclass of class A)
- List of parameters (quantity, type and order)

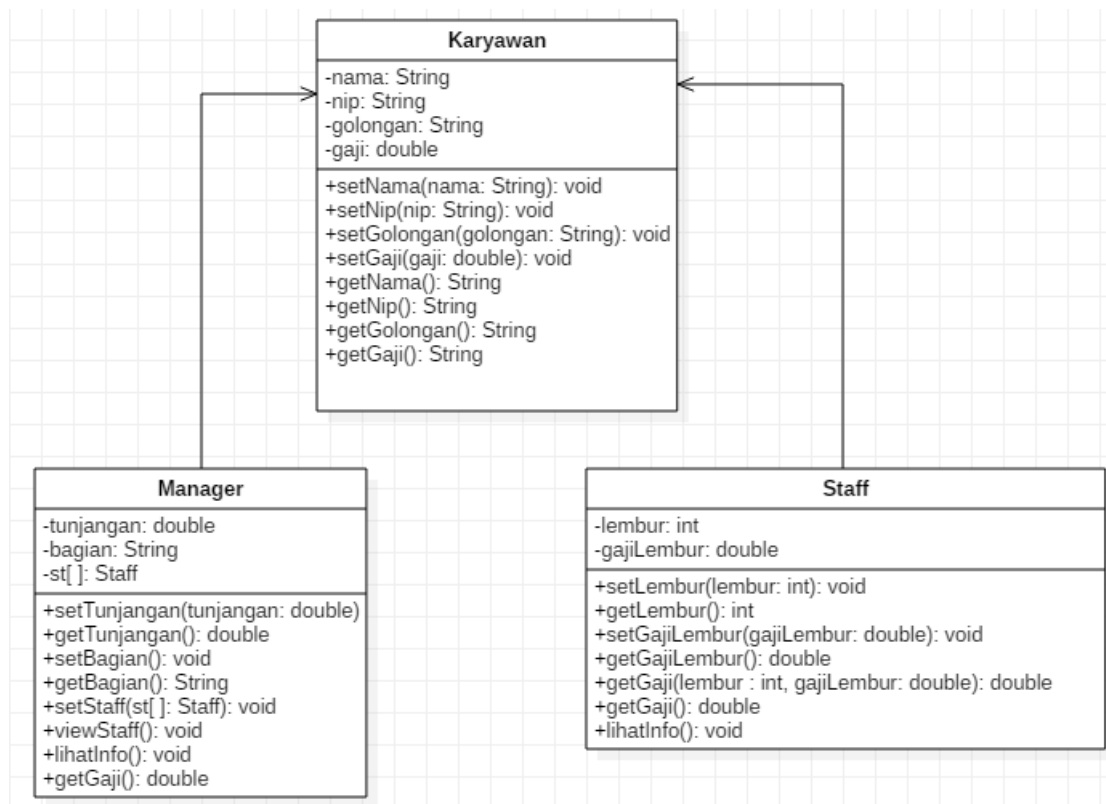
So the method in the parent class is called the overridden method and the method in the subclass is called the overriding method. There are several rules of method in overriding:

- The access mode of the overridden method must be the same or wider than the overridden method.
- Subclasses can only override the superclass method once, there must be no more than one method in the exact same class.
- Overriding method should not throw checked exceptions that are not declared by the overridden method.

3. Practicum

3.1 Experiment 1

For the following example, there are three classes, namely Employee, Manager, and Staff. The Employee Class is a superclass of Manager and Staff where the Subclass of Manager and Staff have different methods to calculate salaries.



3.2 Employees

```

public class Karyawan {

    /**
     * @param args the command line arguments
     */
    // public static void main(String[] args) {
    //     // TODO code application logic here
    private String nama;
    private String nip;
    private String golongan;
    private double gaji;

    public void setNama(String nama)
    {
        this.nama=nama;
    }
    public void setNip(String nip)
    {
        this.nip=nip;
    }
    public void setGolongan(String golongan)
    {
        this.golongan=golongan;

        switch (golongan.charAt(0)) {
            case '1':this.gaji=5000000;
                break;
            case '2':this.gaji=3000000;
                break;
            case '3':this.gaji=2000000;
                break;
            case '4':this.gaji=1000000;
                break;
            case '5':this.gaji=750000;
                break;
        }
    }
    public void setGaji(double gaji)
    {
        this.gaji=gaji;
    }
    public String getNama()
    {
        return nama;
    }
    public String getNip()
    {
        return nip;
    }
    public String getGolongan()
    {
        return golongan;
    }
    public double getGaji()
    {
        return gaji;
    }
}

```

3.3 Staff

```

public class Staff extends Karyawan {
private int lembur;
private double gajiLembur;

public void setLembur(int lembur)
{
    this.lembur=lembur;
}
public int getLembur()
{
    return lembur;
}
public void setGajiLembur(double gajiLembur)
{
    this.gajiLembur=gajiLembur;
}
public double getGajiLembur()
{
    return gajiLembur;
}
public double getGaji(int lembur,double gajiLembur)
{
    return super.getGaji()+lembur*gajiLembur;
}
public double getGaji()
{
    return super.getGaji()+lembur*gajiLembur;
}
public void lihatInfo()
{
    System.out.println("NIP :"+this.getNip());
    System.out.println("Nama :"+this.getNama());
    System.out.println("Golongan :"+this.getGolongan());
    System.out.println("Jml Lembur :"+this.getLembur());
    System.out.printf("Gaji Lembur :%.0f\n", this.getGajiLembur());
    System.out.printf("Gaji :%.0f\n",this.getGaji());
}
}

```

Overloading

Overriding

3.4 Manager

```

public class Manager extends Karyawan {
    private double tunjangan;
    private String bagian;
    private Staff st[];

    public void setTunjangan(double tunjangan)
    {
        this.tunjangan=tunjangan;
    }
    public double getTunjangan()
    {
        return tunjangan;
    }
    public void setBagian(String bagian)
    {
        this.bagian=bagian;
    }
    public String getBagian()
    {
        return bagian;
    }
    public void setStaff(Staff st[])
    {
        this.st=st;
    }

    public void viewStaff()
    {
        int i;
        System.out.println("-----");
        for(i=0;i<st.length;i++)
        {
            st[i].lihatInfo();
        }
        System.out.println("-----");
    }
    public void lihatInfo()
    {
        System.out.println("Manager :"+this.getBagian());
        System.out.println("NIP  :"+this.getNip());
        System.out.println("Nama  :"+this.getNama());
        System.out.println("Golongan :"+this.getGolongan());
        System.out.printf("Tunjangan :%.0f\n",this.getTunjangan());
        System.out.printf("Gaji  :%.0f\n",this.getGaji());
        System.out.println("Bagian  :"+this.getBagian());
        this.viewStaff();
    }
    public double getGaji()
    {
        return super.getGaji()+tunjangan;
    }
}

```

3.5 Main

```

public class Utama {
public static void main(String[] args)
{
    System.out.println("Program Testing Class Manager & Staff");
    Manager man[]=new Manager[2];
    Staff staff1[]=new Staff[2];
    Staff staff2[]=new Staff[3];

    //pembuatan manager

    man[0]=new Manager();
    man[0].setNama("Tedjo");
    man[0].setNip("101");
    man[0].setGolongan("1");
    man[0].setTunjangan(5000000);
    man[0].setBagian("Administrasi");

    man[1]=new Manager();
    man[1].setNama("Atika");
    man[1].setNip("102");
    man[1].setGolongan("1");
    man[1].setTunjangan(2500000);
    man[1].setBagian("Pemasaran");

    staff1[0]=new Staff();
    staff1[0].setNama("Usman");
    staff1[0].setNip("0003");
    staff1[0].setGolongan("2");
    staff1[0].setLembur(10);
    staff1[0].setGajiLembur(10000);

    staff1[1]=new Staff();
    staff1[1].setNama("Anugrah");
    staff1[1].setNip("0005");
    staff1[1].setGolongan("2");
    staff1[1].setLembur(10);
    staff1[1].setGajiLembur(55000);
    man[0].setStaff(staff1);

    staff2[0]=new Staff();
    staff2[0].setNama("Hendra");
    staff2[0].setNip("0004");
    staff2[0].setGolongan("3");
    staff2[0].setLembur(15);
    staff2[0].setGajiLembur(5500);

```

```

staff2[1]=new Staff();
staff2[1].setNama("Arie");
staff2[1].setNip("0006");
staff2[1].setGolongan("4");
staff2[1].setLembur(5);
staff2[1].setGajiLembur(100000);

staff2[2]=new Staff();
staff2[2].setNama("Mentari");
staff2[2].setNip("0007");
staff2[2].setGolongan("3");
staff2[2].setLembur(6);
staff2[2].setGajiLembur(20000);
man[1].setStaff(staff2);

//cetak informasi dari manager + staffnya
man[0].lihatInfo();
man[1].lihatInfo();
}

```

4. Exercise

```

public class PerkalianKu {

    void perkalian(int a, int b){

        System.out.println(a * b);

    }

    void perkalian(int a, int b, int c){

        System.out.println(a * b * c);

    }

    public static void main(String args []){

        PerkalianKu objek = new PerkalianKu();

        objek.perkalian(25, 43);
        objek.perkalian(34, 23, 56);
    }
}

```

4.1 From the source code above, where is the overloading?

4.2 If there is overloading, how many different parameters are there?

```

public class PerkalianKu {
    void perkalian(int a, int b){
        System.out.println(a * b);
    }
    void perkalian(double a, double b){
        System.out.println(a * b);
    }
    public static void main(String args []){
        PerkalianKu objek = new PerkalianKu();
        objek.perkalian(25, 43);
        objek.perkalian(34.56, 23.7);
    }
}

```

4.3 From the source code above, where is the overloading?

4.4 If there is overloading, how many different types of parameters are there?

```

class Ikan{
    public void swim(){
        System.out.println("Ikan bisa berenang");
    }
}
class Piranha extends Ikan{
    public void swim(){
        System.out.println("Piranha bisa makan daging");
    }
}
public class Fish {
    public static void main(String[] args) {
        Ikan a = new Ikan();
        Ikan b = new Piranha();
        a.swim();
        b.swim();
    }
}

```

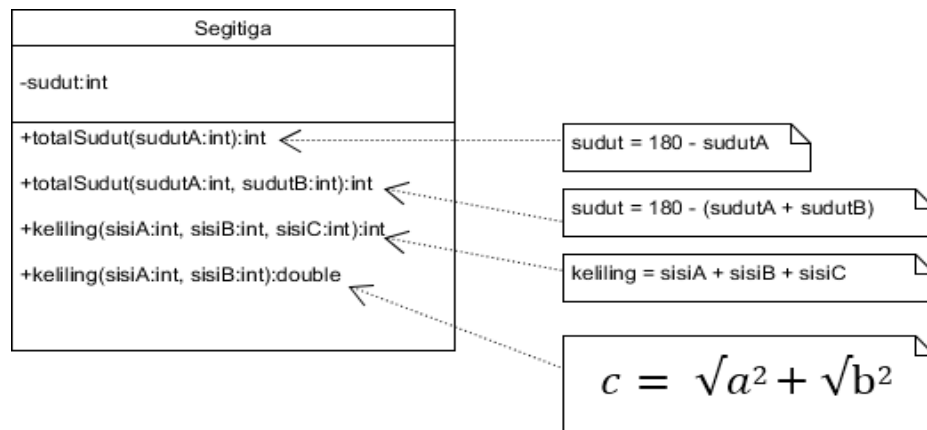
4.5 From the source coding above, where is overriding?

4.6 Describe the sourcoding above if there is overriding?

5. Tasks

5.1 Overloading

Implement the overloading concept in the diagram class below:



5.2 Overriding

Implement the class diagram below using the dynamic method dispatch technique:

